UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM

APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

MODUL 8 – Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)

Nama Mahasiswa	:	Nanik Wijayanti
NIM	:	23051430006
Kelas	:	A2
Tanggal Praktikum	:	17 April 2026
Dosen Pengampu	:	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 8 (Pertemuan 8)
Topik Utama	Asynchronous JavaScript (Fetch API, JSON, REST API Sederhana)
Sub-Topik	Konsep Asynchronous, Fetch API, Promise, Async/Await, dan integrasi REST API eksternal.
Durasi Praktikum	340 Menit
Tanggal Praktikum	17 April 2026
Nama Mahasiswa	Nanik Wijayanti
NIM	23051430006
Kelas	A2

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami perbedaan kode sinkronus dan asinkronus.
2	Mampu menggunakan `fetch()` untuk mengambil data dari server eksternal (API).
3	Memahami format data JSON (JavaScript Object Notation).
4	Menerapkan penanganan error saat request API gagal.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	Console Browser	Browser Modern	Untuk melakukan pengujian logika JavaScript, debugging, serta menampilkan hasil

			eksekusi kode secara langsung selama praktikum
5	Live Server	Ekstensi VS Code	Ekstensi untuk menjalankan server lokal agar perubahan kode web dapat terlihat secara *real-time* di browser

D. DASAR TEORI

Tuliskan ringkasan konsep/teori yang menjadi landasan pada modul ini. Ditulis dengan bahasa sendiri, bukan salinan mentah dari modul. Minimal 3 paragraf.

Pada pertemuan ini, konsep utama yang dipelajari adalah Asynchronous JavaScript, yaitu teknik pemrograman yang memungkinkan suatu proses berjalan tanpa harus menunggu proses lain selesai terlebih dahulu. Dalam aplikasi web modern, pendekatan ini sangat penting karena banyak proses yang melibatkan komunikasi dengan server, seperti pengambilan data atau pengiriman informasi. Jika menggunakan metode sinkron (synchronous), aplikasi akan berhenti sementara hingga proses selesai, yang dapat menyebabkan tampilan menjadi tidak responsif. Oleh karena itu, pendekatan asynchronous digunakan agar aplikasi tetap berjalan lancar dan memberikan pengalaman pengguna yang lebih baik.

Fetch API, yaitu fitur bawaan JavaScript yang digunakan untuk melakukan permintaan data (request) ke server melalui protokol HTTP. Fetch API bekerja dengan konsep Promise, yang merepresentasikan hasil dari suatu operasi yang belum selesai, sehingga memungkinkan penanganan proses secara bertahap menggunakan .then() atau dengan pendekatan yang lebih modern yaitu async/await. Dalam praktiknya, Fetch API sering digunakan untuk mengambil data dari layanan eksternal seperti REST API, yang dalam konteks industri dapat berupa data sensor, database produksi, atau sistem ERP.

Selain itu, konsep penting lainnya adalah format data JSON (JavaScript Object Notation), yang digunakan sebagai standar pertukaran data antara client dan server. JSON memiliki struktur yang ringan dan mudah dibaca, sehingga sangat cocok untuk merepresentasikan data kompleks seperti daftar karyawan atau data produksi. Dalam proses pengambilan data menggunakan Fetch API, data yang diterima biasanya dalam format JSON dan perlu dikonversi menjadi objek JavaScript agar dapat diolah dan ditampilkan ke dalam halaman web. Pemahaman terhadap alur ini memungkinkan pengembangan aplikasi web yang dinamis, terintegrasi, dan mampu berkomunikasi dengan berbagai sumber data eksternal secara efektif.

D.1 Konsep Utama

Fokus utama pada pertemuan 8 ini adalah penggunaan Asynchronous JavaScript, yaitu teknik agar aplikasi web tetap berjalan tanpa harus menunggu proses pengambilan data dari server selesai. Dengan konsep ini, halaman web tidak akan “freeze” saat mengambil data, sehingga lebih responsif bagi pengguna.

Untuk mengambil data dari server digunakan Fetch API, yang memungkinkan aplikasi melakukan request ke API eksternal. Proses ini menggunakan konsep Promise atau async/await, sehingga data dapat diambil dan diolah secara bertahap.

Selain itu, data yang diterima dari server umumnya berbentuk JSON (JavaScript Object Notation). Data ini kemudian dikonversi menjadi objek JavaScript agar dapat ditampilkan di halaman web. Dengan memahami konsep ini, aplikasi dapat terhubung dengan sumber data eksternal secara dinamis.

-

D.2 Keterkaitan dengan Teknik Industri

Materi pada pertemuan 8 sangat relevan bagi mahasiswa Teknik Industri karena dalam era Industri 4.0, sistem industri tidak hanya bergantung pada data lokal, tetapi juga terintegrasi dengan berbagai sumber data eksternal seperti server pusat, IoT, maupun sistem ERP. Kemampuan menggunakan Asynchronous JavaScript dan Fetch API memungkinkan seorang insinyur untuk membangun aplikasi yang dapat mengambil dan menampilkan data secara real-time tanpa mengganggu performa sistem. Hal ini penting untuk mendukung pengambilan keputusan yang cepat dan berbasis data.

Pada Pertemuan 8, konsep pengambilan data dari API menjadi dasar untuk membangun sistem informasi industri yang terhubung. Berikut adalah dua contoh penerapan dalam dunia Teknik Industri:

1. Dashboard Monitoring Produksi Real-Time
   Dengan memanfaatkan Fetch API, aplikasi web dapat mengambil data produksi langsung dari server atau database pusat. Misalnya, data jumlah produksi, status mesin, atau performa operator dapat ditampilkan secara otomatis tanpa perlu input manual. Hal ini membantu supervisor dalam memantau kondisi lini produksi secara real-time dan meningkatkan efisiensi pengawasan.
2. Integrasi Data Supplier atau Inventori
   Melalui API, sistem dapat terhubung dengan data eksternal seperti informasi supplier atau stok bahan baku. Dengan demikian, engineer dapat mengakses data terbaru tanpa harus melakukan input ulang, sehingga mengurangi risiko kesalahan dan meningkatkan akurasi perencanaan produksi.

Secara keseluruhan, penguasaan materi ini memungkinkan mahasiswa Teknik Industri untuk mengembangkan aplikasi web yang dinamis, terintegrasi, dan mampu berkomunikasi dengan berbagai sistem eksternal, sehingga lebih mendekati implementasi nyata di lingkungan industri modern.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

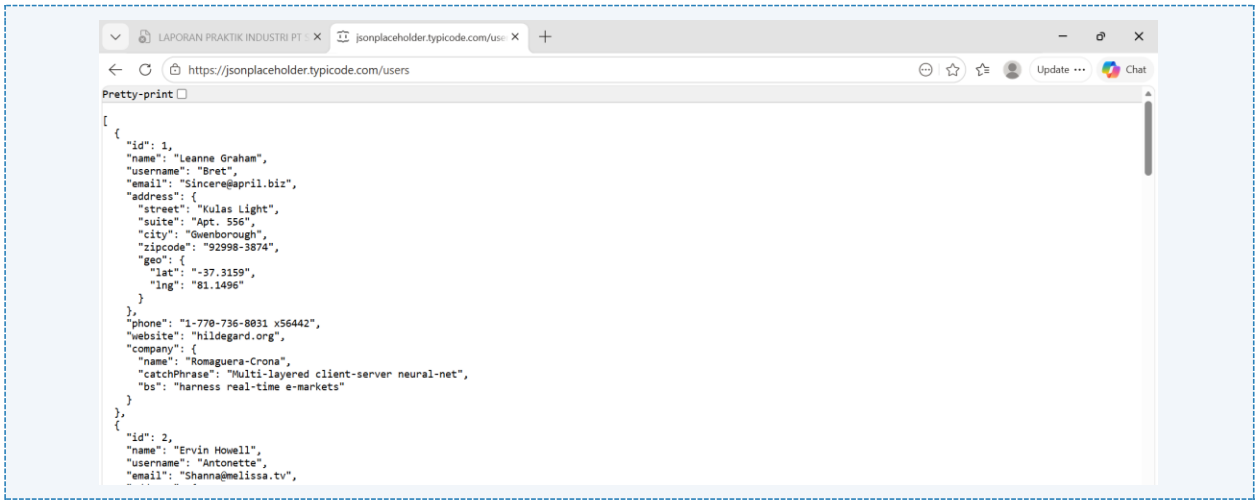
Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

E.1 Langkah 1 – Mengenal JSONPlaceholder

Kita akan menggunakan layanan gratis `https://jsonplaceholder.typicode.com/users` sebagai simulasi database "Karyawan" atau "Agen Supplier".

```
// Endpoint API (Simulasi Database)
const API_URL = "https://jsonplaceholder.typicode.com/users";
```

Screenshot Hasil:



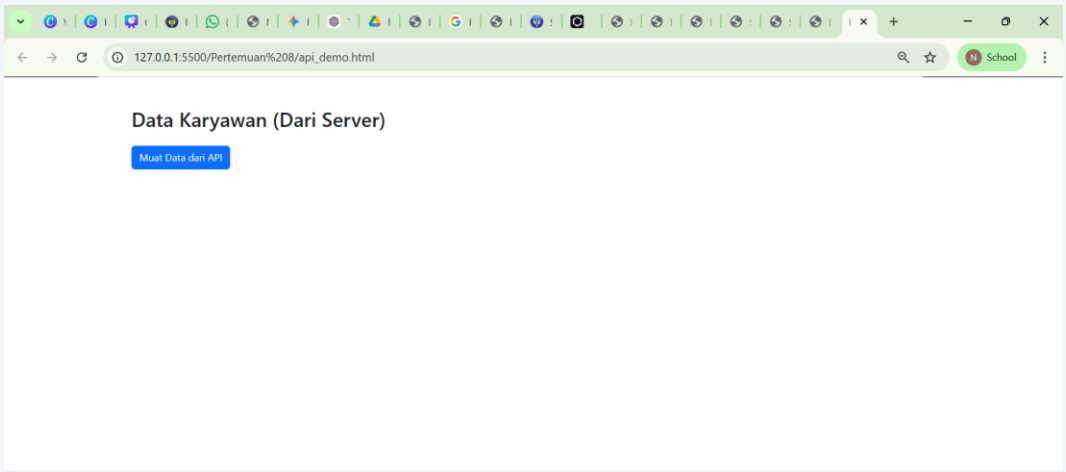
Gambar E.1 – Tampilan Hasil JSONPlaceHolder

E.2 Langkah 2 – Membuat HTML untuk Menampilkan Data

Buat file `api_demo.html`.

```html
<!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <title>Integrasi API Data Karyawan</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.cs
      rel="stylesheet"
    />
  </head>
  <body class="container mt-5">
    <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
    <div class="mb-3">
      <button id="btnLoad" class="btn btn-primary">Muat Data dari API</button>
        id="loading"
        class="spinner-border text-primary d-none"
        role="status"
      ></div>
    </div>
    <div class="row" id="containerKaryawan">
      <!-- Card karyawan akan muncul di sini -->
    </div>
    <script src="api_script.js"></script>
  </body>
</html>
```

Screenshot Hasil:



Gambar E.2 – Tampilan Hasil Membuat HTML

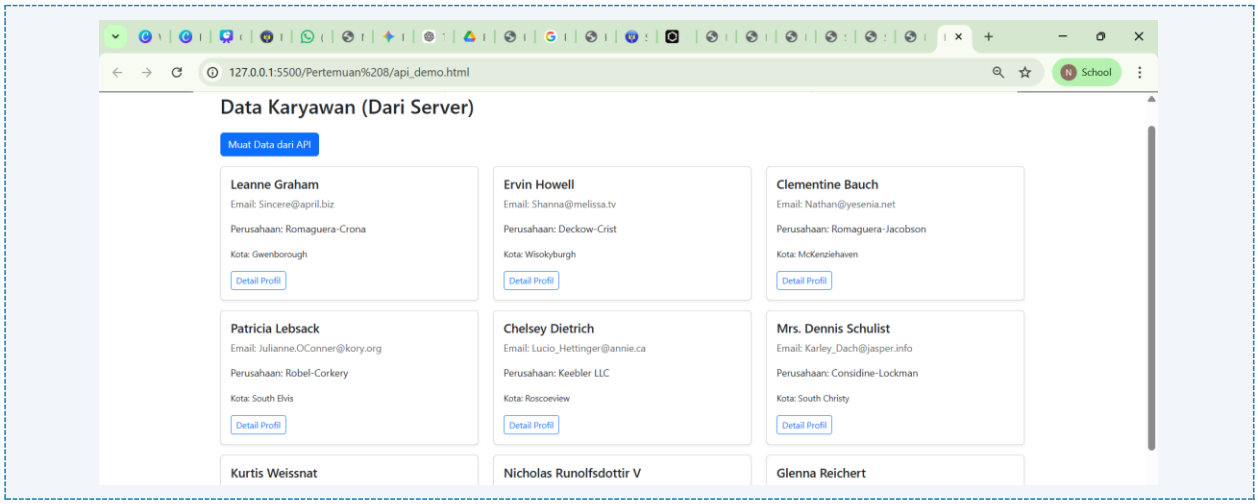
E.3 Langkah 3 – Fetch Data dengan .then() (Promise)

Buat `api_script.js`.

```
 const btnLoad = document.getElementById("btnLoad");
const container = document.getElementById("containerKaryawan");
const loading = document.getElementById("loading");

// Endpoint API (Simulasi Database)
const API_URL = "https://jsonplaceholder.typicode.com/users";

btnLoad.addEventListener("click", function () {
  // Tampilkan loading
  loading.classList.remove("d-none");
  container.innerHTML = ""; // Bersihkan konten lama
  // Fetch Data
  fetch(API_URL)
    .then(function (response) {
      // Cek jika response sukses (kode 200-299)
      if (!response.ok) {
        throw new Error("Gagal mengambil data");
      }
      // Parsing data JSON
      return response.json();
    })
    .then(function (dataKaryawan) {
      // Data berhasil didapat
      console.log(dataKaryawan); // Cek di console browser
      renderData(dataKaryawan);
    })
    .catch(function (error) {
```

```
      // Jika ada error (misal putus internet)
      container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
    })
    .finally(function () {
      // Sembunyikan loading (baik sukses maupun gagal)
      loading.classList.add("d-none");
    });
});
function renderData(data) {
  data.forEach(function (karyawan) {
    // Buat elemen card Bootstrap
    const col = document.createElement("div");
    col.className = "col-md-4 mb-3";

    col.innerHTML = `
            <div class="card h-100 shadow-sm">
                <div class="card-body">
                    <h5 class="card-title">${karyawan.name}</h5>
                    <p class="card-text text-muted">Email:
${karyawan.email}</p>
                    <p class="card-text">Perusahaan:
${karyawan.company.name}</p>
                    <p class="card-text"><small>Kota:
${karyawan.address.city}</small></p>
                    <a href="" class="btn btn-sm btn-outline-primary">Detail
Profil</a>
                </div>
            </div>
        `;

    container.appendChild(col);
  });
}
```

Screenshot Hasil:



Gambar E.3 – Tampilan Hasil Data Karyawan

E.4 Langkah 4 – Fetch dengan Async/Await (Modern Style)

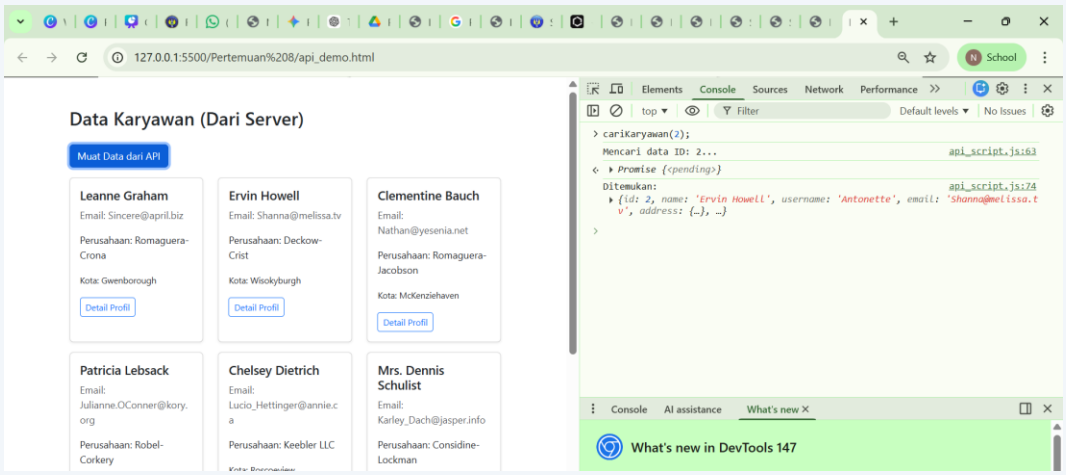
Async/Await membuat kode asinkron terlihat seperti sinkron (lebih mudah dibaca). Mari kita buat fungsi baru untuk mencari karyawan berdasarkan ID.

Tambahkan di `api_script.js`:

```
// Fungsi Async untuk mencari data spesifik
async function cariKaryawan(id) {
  try {
    console.log(`Mencari data ID: ${id}...`);

    const response = await fetch(
      `https://jsonplaceholder.typicode.com/users/${id}`,
    );

    if (!response.ok) {
      throw new Error("Data tidak ditemukan");
    }

    const data = await response.json();
    console.log("Ditemukan:", data);
    alert(`Ditemukan: ${data.name} - bekerja di ${data.company.name}`);
  } catch (error) {
    console.error(error);
    alert(error.message);
  }
}
// Contoh pemanggilan: cariKaryawan(2);
// Anda bisa memanggil fungsi ini lewat console browser untuk tes.
```

[redacted]

Screenshot Hasil:



Gambar E.4 – Tampilan Hasil Pencarian di Console

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1 – Posting Data

Pertanyaan / Instruksi Latihan:

> Buat tombol "Tambah Karyawan Baru". Gunakan method `fetch` dengan opsi `{ method: 'POST', body: ... }` untuk mengirim data dummy ke API. Cek response di console.

Jawaban / Kode Solusi:

Menambahkan kode di JS.

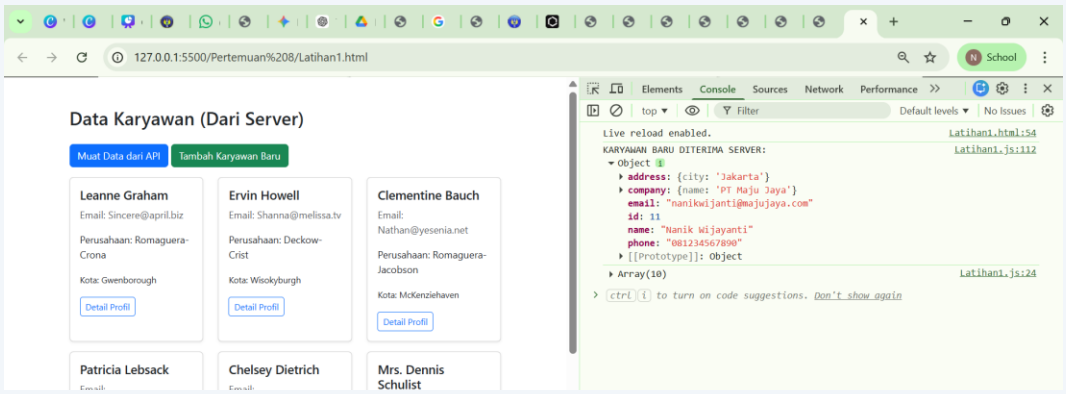
```
// Latihan 1: POST Data Karyawan Baru
const btnTambah = document.getElementById("btnTambah");

btnTambah.addEventListener("click", function () {
  // Data dummy karyawan baru
  const karyawanBaru = {
    name: "Nanik Wijayanti",
    email: "nanikwijanti@majujaya.com",
    phone: "081234567890",
    company: { name: "PT Maju Jaya" },
    address: { city: "Jakarta" },
```

```
  };

  // POST ke API
  fetch(API_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(karyawanBaru),
  })
    .then(function (response) {
      if (!response.ok) {
        throw new Error(`POST gagal: ${response.status}`);
      }
      return response.json();
    })
    .then(function (data) {
      console.log("KARYAWAN BARU DITERIMA SERVER:", data);
      alert(`Sukses! ID: ${data.id}\nName: ${data.name}`);

      // Auto reload data untuk lihat hasil
      document.getElementById("btnLoad").click();
    })
    .catch(function (error) {
      console.error("POST ERROR:", error);
      alert("Gagal tambah karyawan: " + error.message);
    });
});

<button id="btnTambah" class="btn btn-success">Tambah Karyawan Baru</button>
```

Menambahkan kode di HTML.

```
<button id="btnTambah" class="btn btn-success">Tambah Karyawan Baru</button>
```

Screenshot Hasil:



Gambar F.1 – Tampilan Hasil Menambah Karyawan Baru

F.2 Latihan 2 – Filter Array

Pertanyaan / Instruksi Latihan:

> Sebelum data dirender ke layar dengan `forEach`, lakukan filter terlebih dahulu. Tampilkan HANYA karyawan yang tinggal di kota yang mengandung huruf "s" (misal: "Southside", "London"). Petunjuk: Gunakan `data.filter(...)` sebelum `data.forEach(...)`.

Jawaban / Kode Solusi:

Tambahkan kode pada HTML:

```
<!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <title>Integrasi API Data Karyawan - Nanik Wijayanti (23051430006)</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.cs
      rel="stylesheet"
    />
  </head>

  <body class="container mt-5">
    <div class="text-center mb-4">
      <h4 class="text-primary">Praktikum Integrasi API Data Karyawan</h4>
      <h5 class="text-muted">Oleh: <strong>Nanik Wijayanti</strong></h5>
      <h6 class="text-secondary">NIM: <strong>23051430006</strong></h6>
    </div>

    <h2 class="mb-4">Data Karyawan (Dari Server)</h2>
    <div class="mb-3">
      <button id="btnLoad" class="btn btn-primary me-2">Muat Semua Data</button>
      <button id="btnFilter" class="btn btn-warning me-2">
        Filter Kota 's'
      </button>
      <button id="btnTambah" class="btn btn-success">
```

```
        Tambah Karyawan Baru
      </button>
      <div
        id="loading"
        class="spinner-border text-primary d-none ms-2"
        role="status"
      ></div>
    </div>

    <!-- Info Filter -->
    <div id="infoFilter" class="alert alert-info d-none mb-3">
      <strong>Filter aktif:</strong> Menampilkan hanya karyawan dari kota yang
      mengandung huruf "s"
    </div>

    <div class="row" id="containerKaryawan">
      <!-- Card karyawan akan muncul di sini -->
    </div>
    <script src="Latihan2.js"></script>
  </body>
</html>
```

Tambahkan kode pada JS:

```
const btnLoad = document.getElementById("btnLoad");
const btnFilter = document.getElementById("btnFilter");
const btnTambah = document.getElementById("btnTambah");
const container = document.getElementById("containerKaryawan");
const loading = document.getElementById("loading");
const infoFilter = document.getElementById("infoFilter");

// Endpoint API
const API_URL = "https://jsonplaceholder.typicode.com/users";

// Variable global untuk status filter
let isFilterActive = false;

btnLoad.addEventListener("click", function () {
  loading.classList.remove("d-none");
  container.innerHTML = "";
  fetch(API_URL)
    .then(function (response) {
      if (!response.ok) {
        throw new Error("Gagal mengambil data");
      }
```

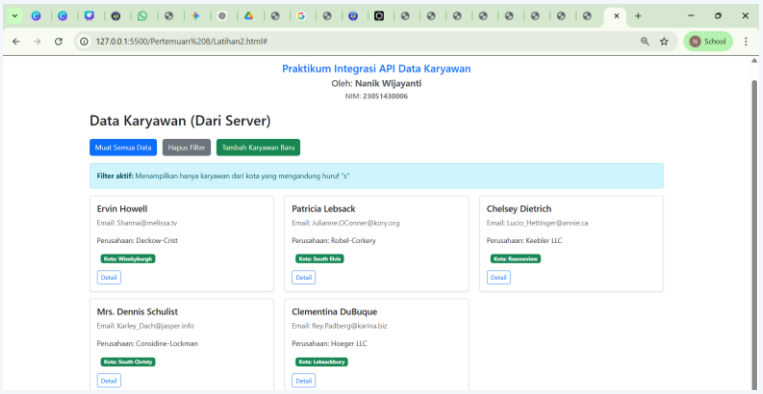
```
      return response.json();
    })
    .then(function (dataKaryawan) {
      console.log("Total data:", dataKaryawan.length);
      renderData(dataKaryawan);
    })
    .catch(function (error) {
      container.innerHTML = `<div class="alert alert-danger">Error:
${error.message}</div>`;
    })
    .finally(function () {
      loading.classList.add("d-none");
    });
});

// ← BARU: Tombol Filter
btnFilter.addEventListener("click", function () {
  isFilterActive = !isFilterActive; // Toggle filter

  if (isFilterActive) {
    btnFilter.textContent = "Hapus Filter";
    btnFilter.className = "btn btn-secondary me-2";
    infoFilter.classList.remove("d-none");
    console.log('Filter AKTIF - Hanya kota mengandung "s"');
  } else {
    btnFilter.textContent = "Filter Kota 's'";
    btnFilter.className = "btn btn-warning me-2";
    infoFilter.classList.add("d-none");
    console.log("Filter DINONAKTIFKAN");
  }

  // Reload data dengan status filter baru
  btnLoad.click();
});

// Fungsi tambah karyawan (tetap sama)
btnTambah.addEventListener("click", function () {
  tambahKaryawanBaru();
});

function tambahKaryawanBaru() {
  const karyawanBaru = {
    name: "Nanik Wijayanti",
    email: "nanikwijayani@majujaya.com",
    phone: "081234567890",
    website: "majujaya.com",
```

```
    company: {
      name: "PT Maju Jaya",
      catchPhrase: "Membangun masa depan digital",
      bs: "Digital solutions provider",
    },
    address: {
      street: "Jl. Nusa Indah No. 179",
      suite: "Lantai 5",
      city: "Semarang", // ← Mengandung "s"
      zipcode: "34521",
    },
  };

  loading.classList.remove("d-none");

  console.log("Mengirim data POST:", karyawanBaru);

  fetch(API_URL, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(karyawanBaru),
  })
    .then(function (response) {
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      return response.json();
    })
    .then(function (responseData) {
      console.log("POST berhasil!", responseData);
      alert(`Karyawan berhasil ditambahkan!\nID: ${responseData.id}`);
    })
    .catch(function (error) {
      console.error("Error POST:", error);
      alert(`Gagal: ${error.message}`);
    })
    .finally(function () {
      loading.classList.add("d-none");
    });
}

// ← MODIFIKASI: Fungsi renderData dengan FILTER
function renderData(data) {
  console.log("Rendering data...", data.length, "items");
```

```
  let dataToRender = data;

  //FILTER: Hanya kota yang mengandung huruf "s"
  if (isFilterActive) {
    dataToRender = data.filter(function (karyawan) {
      // Cek apakah kota mengandung huruf "s" (case-insensitive)
      return karyawan.address.city.toLowerCase().includes("s");
    });
    console.log(
      `Filter diterapkan: ${dataToRender.length}/${data.length} karyawan dari kota
dengan huruf "s"`,
    );
  }

  // Render data yang sudah difilter
  dataToRender.forEach(function (karyawan) {
    const col = document.createElement("div");
    col.className = "col-md-4 mb-3";

    // Tambah badge untuk kota yang difilter
    const badgeFilter = isFilterActive
      ? `<span class="badge bg-success ms-2">Kota:
${karyawan.address.city}</span>`
      : `<small>Kota: ${karyawan.address.city}</small>`;

    col.innerHTML = `
<div class="card h-100 shadow-sm">
<div class="card-body">
<h5 class="card-title">${karyawan.name}</h5>
<p class="card-text text-muted">Email: ${karyawan.email}</p>
<p class="card-text">Perusahaan: ${karyawan.company.name}</p>
<p class="card-text">${badgeFilter}</p>
<a href="#" class="btn btn-sm btn-outline-primary"
onclick="cariKaryawan(${karyawan.id})">Detail</a>
</div>
</div>
`;
    container.appendChild(col);
  });

  // Update info jumlah data
  if (dataToRender.length === 0) {
    container.innerHTML = `
            <div class="col-12">
                <div class="alert alert-warning text-center">
```

```
                    <h5>Tidak ada data</h5>
                    ${isFilterActive ? '(Semua kota tidak mengandung huruf "s")' :
'(Klik "Muat Semua Data" terlebih dahulu)'}
                </div>
            </div>
        `;
  }
}

// Fungsi cariKaryawan (diperbaiki agar bisa dipanggil dari HTML onclick)
function cariKaryawan(id) {
  fetch(`https://jsonplaceholder.typicode.com/users/${id}`)
    .then((response) => {
      if (!response.ok) throw new Error("Data tidak ditemukan");
      return response.json();
    })
    .then((data) => {
      console.log("Detail:", data);
      alert(
        `${data.name}\n${data.email}\n${data.company.name}\n${data.address.city}`,
      );
    })
    .catch((error) => {
      console.error(error);
      alert(error.message);
    });
}
```

Screenshot Hasil:



Gambar F.2 – Tampilan Hasil Filter “S”

F.3 Tugas Proyek Mini – Integrasi Data Produksi

Deskripsi proyek mini:

> Anggap API `https://jsonplaceholder.typicode.com/posts` adalah database "Laporan Insiden".
> a. Tampilkan daftar 10 laporan insiden terbaru (Title = Judul Insiden, Body = Deskripsi).
> b. Di setiap card, tambahkan tombol "Tindak Lanjut". Saat diklik, `alert()` menampilkan "Tiket ID sedang diproses oleh Tim Maintenance".

Penjelasan pendekatan/solusi yang digunakan:

[Tuliskan penjelasan Anda tentang cara menyelesaikan proyek mini ini]

Dalam menyelesaikan proyek mini integrasi data produksi ini, saya menggunakan pendekatan Asynchronous JavaScript dengan Fetch API untuk mengambil data dari server dan menampilkannya secara dinamis ke dalam halaman web. Data diambil dari API jsonplaceholder.typicode.com/posts yang diasumsikan sebagai database laporan insiden. Proses pengambilan data dilakukan menggunakan ‘fetch()’ dengan metode GET, kemudian data yang diterima dalam format JSON diproses menggunakan ‘.then()’ sebelum ditampilkan ke antarmuka.

Selanjutnya, saya membatasi data yang ditampilkan hanya 10 laporan insiden pertama menggunakan metode ’.slice(0, 10)’. Untuk membuat data lebih relevan dengan konteks industri, saya mengganti isi data API (lorem ipsum) dengan data insiden produksi yang lebih realistis menggunakan metode ’.map()’. Data hasil olahan tersebut disimpan ke dalam variabel global agar dapat digunakan kembali dalam proses pencarian dan filter tanpa harus melakukan fetch ulang.

Untuk menampilkan data, saya membuat fungsi ‘renderListKeLayar()’ yang bertugas melakukan manipulasi DOM dengan membuat elemen card Bootstrap secara dinamis. Setiap card menampilkan ID, judul, deskripsi, serta informasi lokasi produksi berdasarkan logika ID (ganjil = Line A, genap = Line B). Selain itu, ditambahkan tombol **“Tindak Lanjut”** yang menggunakan event onclick untuk menampilkan ‘alert()’ sebagai simulasi proses penanganan tiket oleh tim maintenance.

Saya juga menambahkan fitur pencarian dan filter real-time menggunakan event input dan change. Data difilter berdasarkan kata kunci (judul/deskripsi) dan lokasi produksi menggunakan metode ‘.filter()’, sehingga tampilan dapat diperbarui secara langsung tanpa reload halaman. Selain itu, terdapat fitur tambahan seperti tambah data (POST) menggunakan Fetch API dan hapus data (filter array) untuk mensimulasikan operasi CRUD sederhana.

Kode di HTML:

```html
<!doctype html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Sistem Pelaporan Insiden</title>
    <link
      href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.cs
      rel="stylesheet"
    />
    <link rel="stylesheet" href="insiden.css" />
  </head>
  <body class="container mt-5">
    <div
      class="d-flex justify-content-between align-items-center mb-4 border-bottom
pb-3"
    >
      <div>
        <h2 class="text-dark fw-bold mb-0">Laporan Insiden Produksi</h2>
        <span class="badge bg-secondary mt-2"
          >Penanggung Jawab: Nanik Wijayanti</span
        >
      </div>
      <div>
        <button id="btnLoadInsiden" class="btn btn-outline-secondary me-2">
          Muat Data Server
        </button>
        <button id="btnTambahInsiden" class="btn btn-corporate">
          Buat Tiket Baru
        </button>
      </div>
    </div>

    <div class="row mb-4 bg-white p-3 shadow-sm rounded-3 mx-0 border">
      <div class="col-md-8">
        <label class="form-label fw-bold text-secondary small"
          >Pencarian Kata Kunci</label
        >
        <input
          type="text"
          id="inputSearch"
          class="form-control"
          placeholder="Cari berdasarkan judul atau deskripsi tiket..."
        />
      </div>
      <div class="col-md-4">
```

```
        <label class="form-label fw-bold text-secondary small"
          >Filter Lokasi</label
        >
        <select id="selectFilter" class="form-select">
          <option value="all">Semua Line Produksi</option>
          <option value="lineA">Line Produksi A (ID Ganjil)</option>
          <option value="lineB">Line Produksi B (ID Genap)</option>
        </select>
      </div>
    </div>

    <div id="loading" class="text-center d-none my-5">
      <div class="spinner-border text-primary" role="status"></div>
      <p class="text-muted mt-2 fw-semibold">Memproses data...</p>
    </div>

    <div class="row" id="containerInsiden"></div>

    <script src="insiden.js"></script>
  </body>
</html>
```

Kode di JS:

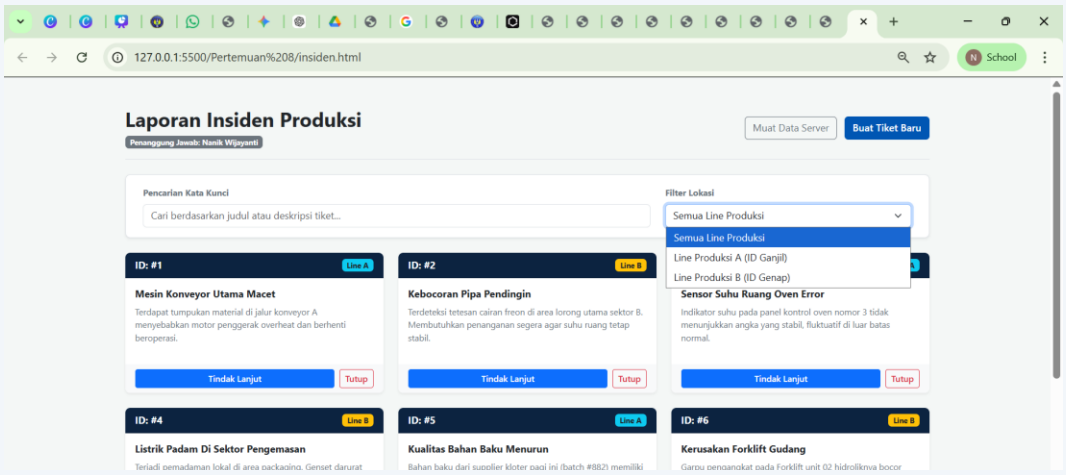
```
const btnLoad = document.getElementById("btnLoadInsiden");
const btnTambah = document.getElementById("btnTambahInsiden");
const container = document.getElementById("containerInsiden");
const loading = document.getElementById("loading");

const inputSearch = document.getElementById("inputSearch");
const selectFilter = document.getElementById("selectFilter");

// Endpoint API
const API_URL = "https://jsonplaceholder.typicode.com/posts";

// Variabel Global untuk menyimpan data
let dataInsidenGlobal = [];

const laporanReal = [
  {
    title: "Mesin Konveyor Utama Macet",
    body: "Terdapat tumpukan material di jalur konveyor A menyebabkan motor
penggerak overheat dan berhenti beroperasi.",
  },
  {
    title: "Kebocoran Pipa Pendingin",
    body: "Terdeteksi tetesan cairan freon di area lorong utama sektor B.
Membutuhkan penanganan segera agar suhu ruang tetap stabil.",
  },
  {
    title: "Sensor Suhu Ruang Oven Error",
    body: "Indikator suhu pada panel kontrol oven nomor 3 tidak menunjukkan
angka yang stabil, fluktuatif di luar batas normal.",
  },
  {
    title: "Listrik Padam di Sektor Pengemasan",
    body: "Terjadi pemadaman lokal di area packaging. Genset darurat sudah
aktif namun beberapa mesin mati mendadak.",
  },
  {
    title: "Kualitas Bahan Baku Menurun",
    body: "Bahan baku dari supplier kloter pagi ini (batch #882) memiliki
tingkat kelembapan di atas standar kualitas.",
  },
  {
    title: "Kerusakan Forklift Gudang",
    body: "Garpu pengangkat pada Forklift unit 02 hidroliknya bocor sehingga
tidak bisa mengangkat beban maksimal.",
  },
  {
```

```
    title: "Jaringan Internet Terputus",
    body: "Koneksi ke server pusat terputus sejak jam 08:00 pagi. Pembaruan
data stok barang tidak bisa dilakukan secara realtime.",
  },
  {
    title: "Kebakaran Kecil di Ruang Genset",
    body: "Terjadi korsleting ringan yang menimbulkan percikan api. Api sudah
dipadamkan pakai APAR, tapi butuh perbaikan kabel.",
  },
  {
    title: "Pintu Otomatis Loading Dock Rusak",
    body: "Sensor gerak pintu gerbang pemuatan barang tidak merespon, pintu
harus dibuka secara manual.",
  },
  {
    title: "Sistem Barcode Scanner Gagal Baca",
    body: "Scanner di line B tidak mengenali barcode produk baru, menyebabkan
antrean panjang di jalur quality control.",
  },
];

// 1. FUNGSI READ: Mengambil Data dari Server API
btnLoad.addEventListener("click", function () {
  loading.classList.remove("d-none");
  container.innerHTML = "";

  fetch(API_URL)
    .then((response) => {
      if (!response.ok) throw new Error("Gagal terhubung ke server ERP");
      return response.json();
    })
    .then((dataPosts) => {
      // Ambil 10 data dari API
      let sepuluhLaporan = dataPosts.slice(0, 10);

      // KODE SULAP: Mengganti isi Lorem Ipsum dengan data bahasa Indonesia
kita
      sepuluhLaporan = sepuluhLaporan.map((insiden, index) => {
        return {
          id: insiden.id,
          title: laporanReal[index].title,
          body: laporanReal[index].body,
        };
      });

      // Simpan ke variabel global
```

```
      dataInsidenGlobal = sepuluhLaporan;

      // Tampilkan ke layar
      renderListKeLayar(dataInsidenGlobal);
    })
    .catch((error) => {
      container.innerHTML = `<div class="alert alert-danger shadow-
sm"><strong>Sistem Error:</strong> ${error.message}</div>`;
    })
    .finally(() => {
      loading.classList.add("d-none");
    });
});

// 2. FUNGSI RENDER: Menggambar Kartu ke Layar HTML
function renderListKeLayar(arrayData) {
  container.innerHTML = "";

  if (arrayData.length === 0) {
    container.innerHTML = `<div class="col-12 text-center text-muted mt-
4"><em>Tidak ada tiket insiden yang sesuai dengan pencarian.</em></div>`;
    return;
  }

  arrayData.forEach((insiden) => {
    const lokasi = insiden.id % 2 === 0 ? "Line B" : "Line A";
    const badgeColor =
      insiden.id % 2 === 0 ? "bg-warning text-dark" : "bg-info text-dark";

    const col = document.createElement("div");
    col.className = "col-md-6 col-lg-4 mb-4";
    col.id = `tiket-${insiden.id}`;

    col.innerHTML = `
            <div class="card h-100 shadow-sm border-0 rounded-3">
                <div class="card-header card-header-corporate d-flex justify-
content-between align-items-center rounded-top-3" style="background-color:
#0c2340; color: white;">
                    <span class="fw-bold">ID: #${insiden.id}</span>
                    <span class="badge ${badgeColor}">${lokasi}</span>
                </div>
                <div class="card-body bg-white flex-column d-flex">
                    <h6 class="card-title text-capitalize fw-bold text-
dark">${insiden.title}</h6>
                    <p class="card-text text-secondary small mb-
3">${insiden.body}</p>
```

```
                </div>
                <div class="card-footer bg-light border-top-0 d-flex gap-2
rounded-bottom-3">
                    <button class="btn btn-sm btn-primary flex-fill fw-
semibold" onclick="tindakLanjut(${insiden.id})">Tindak Lanjut</button>
                    <button class="btn btn-sm btn-outline-danger fw-semibold"
onclick="tutupTiket(${insiden.id})">Tutup</button>
                </div>
            </div>
        `;
    container.appendChild(col);
  });
}

// 3. FUNGSI PENCARIAN & FILTER REALTIME
function jalankanSearchDanFilter() {
  const kataKunci = inputSearch.value.toLowerCase();
  const filterLokasi = selectFilter.value;

  const hasilFilter = dataInsidenGlobal.filter((insiden) => {
    const cocokTeks =
      insiden.title.toLowerCase().includes(kataKunci) ||
      insiden.body.toLowerCase().includes(kataKunci);

    let cocokLokasi = false;
    if (filterLokasi === "all") {
      cocokLokasi = true;
    } else if (filterLokasi === "lineA" && insiden.id % 2 !== 0) {
      cocokLokasi = true;
    } else if (filterLokasi === "lineB" && insiden.id % 2 === 0) {
      cocokLokasi = true;
    }

    return cocokTeks && cocokLokasi;
  });

  renderListKeLayar(hasilFilter);
}

inputSearch.addEventListener("input", jalankanSearchDanFilter);
selectFilter.addEventListener("change", jalankanSearchDanFilter);

// 4. FUNGSI ACTION, CREATE (TAMBAH), & DELETE (HAPUS)

function tindakLanjut(id) {
```

```
  alert(`[SISTEM ERP] Tiket ID #${id} sedang diproses oleh Tim
Maintenance.`);
}

btnTambah.addEventListener("click", function () {
  const tiketBaru = {
    title: "Tumpahan Bahan Kimia (Data Baru)",
    body: "Drum penyimpanan bahan kimia H2O2 terguling. Area sudah diisolasi,
butuh tim kebersihan khusus segera.",
    userId: 1,
  };
  loading.classList.remove("d-none");

  fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(tiketBaru),
  })
    .then((res) => res.json())
    .then((data) => {
      alert(`[SUKSES] Tiket baru dibuat dengan ID #${data.id}`);
      dataInsidenGlobal.unshift(data);
      renderListKeLayar(dataInsidenGlobal);
    })
    .catch((error) => alert("Gagal membuat tiket: " + error.message))
    .finally(() => loading.classList.add("d-none"));
});

function tutupTiket(id) {
  if (confirm(`Apakah Anda yakin ingin menutup tiket #${id}?`)) {
    dataInsidenGlobal = dataInsidenGlobal.filter(
      (insiden) => insiden.id !== id,
    );
    renderListKeLayar(dataInsidenGlobal);
  }
}
```



Gambar F.3 – Tampilan Hasil Proyek Mini

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Setelah menjalankan aplikasi pelaporan insiden berbasis API, hasil yang ditampilkan sudah sesuai dengan materi pada pertemuan 8, khususnya terkait penggunaan Asynchronous JavaScript dan Fetch API. Data dari server berhasil diambil dan ditampilkan ke dalam bentuk card tanpa mengganggu tampilan halaman, sehingga menunjukkan bahwa konsep asynchronous berjalan dengan baik.

Pada proses pengambilan data, penggunaan ‘fetch()’ dengan metode ‘.then()’ sudah sesuai dengan teori Promise, di mana data diambil secara bertahap mulai dari request, response, hingga konversi ke format JSON. Selain itu, terdapat pengecekan response.ok untuk menangani error, yang menunjukkan bahwa aplikasi sudah menerapkan konsep error handling sesuai materi.

Selanjutnya, data yang diterima dalam format JSON berhasil diproses menggunakan metode ‘.slice()’ untuk membatasi jumlah data dan ‘.map()’ untuk menyesuaikan isi data dengan konteks industri. Hal ini sesuai dengan teori bahwa data dari API dapat dimodifikasi sebelum ditampilkan agar lebih relevan dengan kebutuhan pengguna.

Pada bagian tampilan, data berhasil dirender secara dinamis menggunakan manipulasi DOM melalui fungsi ‘renderListKeLayar()’. Setiap data ditampilkan dalam bentuk card Bootstrap lengkap dengan informasi ID, judul, dan deskripsi. Selain itu, tombol “Tindak Lanjut” juga berfungsi dengan baik menggunakan event handler yang menampilkan ‘alert()’, sesuai dengan instruksi praktikum.

Fitur tambahan seperti pencarian dan filter menggunakan ‘.filter()’ juga berjalan sesuai teori, di mana data dapat difilter secara real-time tanpa perlu mengambil ulang data dari server. Secara keseluruhan, aplikasi ini telah berhasil menerapkan konsep asynchronous, Fetch API, pengolahan data JSON, serta manipulasi DOM, dan hasil yang diperoleh sudah sesuai dengan teori pada modul pertemuan 8.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Tombol "Muat Data Server" tidak bisa diklik	Mengecek kembali apakah id pada HTML (btnLoadInsiden) sudah sesuai dengan yang dipanggil di JavaScript (getElementById). Selain itu, memastikan file JavaScript sudah terhubung dengan benar dan tidak ada error di Console (F12).
Data dari API tidak muncul di tampilan	Memastikan fungsi fetch() berjalan dengan benar dan endpoint API sesuai. Juga menambahkan pengecekan response.ok serta melihat error pada Console untuk mengetahui penyebab kegagalan.
Data tidak bisa difilter atau dicari	Memastikan data sudah disimpan ke variabel global (dataInsidenGlobal) sebelum proses filter dijalankan, sehingga fungsi .filter() dapat bekerja dengan baik.
Error saat menampilkan data	Mengecek struktur data JSON dan memastikan properti seperti title dan body sesuai dengan yang dipanggil di kode.

G.3 Kaitan dengan Penerapan di Industri

Keterampilan pemrograman pada modul ini sangat penting dalam mendukung sistem industri modern yang terintegrasi dan berbasis data real-time. Dalam Teknik Industri, kemampuan mengambil dan menampilkan data dari server secara langsung menjadi kunci dalam monitoring, pengendalian, dan pengambilan keputusan yang cepat. Berikut adalah tiga skenario penerapan materi Pertemuan 8 di dunia industri:

1. Dashboard Monitoring Produksi Real-Time
   Dengan menggunakan Fetch API, aplikasi dapat mengambil data produksi langsung dari server pusat atau sistem ERP. Misalnya, supervisor dapat melihat jumlah output, status mesin, atau performa operator secara real-time tanpa perlu input manual. Hal ini membantu meningkatkan efisiensi pengawasan dan respons terhadap masalah di lini produksi.
2. Sistem Pelaporan Insiden Terintegrasi
   Melalui integrasi API, laporan insiden yang dibuat oleh operator dapat langsung tersimpan dan diakses oleh tim terkait seperti maintenance atau quality control. Tombol seperti "Tindak Lanjut" dapat digunakan sebagai simulasi proses penanganan tiket, sehingga alur komunikasi antar departemen menjadi lebih cepat dan terstruktur.
3. Integrasi Data Supplier dan Inventori
   Dengan memanfaatkan API, sistem dapat terhubung dengan data eksternal seperti stok bahan baku atau informasi supplier secara otomatis. Hal ini memudahkan perencanaan produksi dan mengurangi risiko kesalahan akibat input manual, sehingga keputusan yang diambil menjadi lebih akurat dan berbasis data terbaru.

Secara keseluruhan, penguasaan materi ini memungkinkan pengembangan aplikasi web yang dinamis, terintegrasi dengan berbagai sistem, serta mampu mendukung kebutuhan industri modern yang berbasis digital dan real-time

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Saya belajar bahwa Asynchronous JavaScript memungkinkan aplikasi web tetap berjalan tanpa harus menunggu proses pengambilan data dari server selesai, sehingga tampilan tetap responsif dan tidak “freeze”.
2	Penggunaan Fetch API memudahkan dalam mengambil data dari server (API) secara langsung, sehingga aplikasi dapat terhubung dengan sumber data eksternal tanpa perlu database sendiri.
3	Data dari server yang berbentuk JSON dapat diolah menggunakan JavaScript, seperti membatasi jumlah data (slice) dan menyesuaikan isi data (map) sebelum ditampilkan ke pengguna.
4	Saya memahami bagaimana melakukan manipulasi DOM untuk menampilkan data secara dinamis dalam bentuk card, serta menambahkan interaksi seperti tombol “Tindak Lanjut” menggunakan event handler.
5	Dengan tambahan fitur search dan filter, aplikasi menjadi lebih interaktif dan mampu mengelola data secara efisien tanpa perlu reload halaman.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
	TOTAL NILAI AKHIR	100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Nanik Wijayanti NIM: 23051430006	_____ Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm | Bawah: 3 cm | Kiri: 4 cm | Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → … → Laporan Modul 12 → Laporan Modul 14 & 15.

c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```
PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)
```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib temui dosen untuk klarifikasi
Tidak dikumpulkan	**Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS**

— Selamat mengerjakan —